

QA Start

Регресс · безопасность · производительность · готовность к запуску

Итог: **NO-GO** до устранения обхода прогресса и проверки/обновления уязвимых зависимостей.

Проверялся репозиторий проекта на коммите 551d782 (релиз v0.4.1), Supabase-проект и безопасные production smoke-проверки. Аудит выполнен 5 октября 2026 года.

Область	Результат
Регресс	TypeScript, lint, production build и 50 Node unit-тестов — успешно.
Python / Playwright	68 passed, 3 skipped, 1 failed, 1 error; полный прогон 243.8 с.
Production smoke	Публичные страницы, auth, CORS и REST прошли; проверка приватного Storage пропущена без service-role секрета.
База и доступы	Все 19 таблиц схемы public имеют RLS. Найдено разрешение ученику самому записывать завершение урока.
Зависимости	npm audit --omit=dev: 31 advisories; среди high — DOMPurify, используемый при выводе HTML урока.
Нагрузка	Не запускалась: подтверждённой изолированной staging/test-БД нет; production под нагрузку не подвергался.

Решение по запуску

В текущем состоянии запускать рискованно: self-service запись в lesson_progress обходит учебные ограничения и позволяет открыть итоговый тест. Одновременно production-зависимости содержат high advisories. Полный browser-run также поймал страницу «Что-то пошло не так» при загрузке входа ученика; одиночный повтор прошёл, поэтому это плавающий, но не закрытый сбой.

Покрытие автотестами

Матрица отражает найденные типы тестов и что реально было исполнено. UI-тесты итогового теста, которые расходуют попытку, намеренно не запускались на рабочей production-учётке.

Функциональная область	Нынешнее покрытие	Оценка / пробел
Публичные страницы / вход	Playwright: landing, мобильный viewport, форма и валидация; smoke HTTP.	Среднее. Один full-run поймал generic error screen на /auth; isolated повтор прошёл.
Уроки / блоки / импорт	Unit: экспорт/импорт и схемы; UI: редактор, preview/import, responsive admin.	Хорошее на формат/клиенте; фактическую запись/rollback проверять только на staging.
Прогресс и доступность уроков	Unit: очередь сохранения/offline retry; SQL-контракт миграций.	Критический пробел: live RLS разрешает self-write completed; добавлен failing regression test.
Вопросы / SQL-практика	Unit: банк итогового теста, sandbox SQL whitelist, CRUD verifier и импорт-конфиги.	Хорошо в unit; нет отдельного API negative test от ученика к чужому block_id.
Домашние задания	Unit: статусы / SQL completion; admin UI smoke очереди и отзывчивости.	Частично. Нет безопасного E2E submit → review → approved/rejected на disposable backend.
Итоговый тест	Unit: 90 вопросов, schema validation, completion; UI settings smoke.	Механика selection/timer/attempt mutation покрыта условно, 3 mutation-теста пропущены.
Админка / роли	UI: students, lesson editor, groups/meetings; миграции и live policies.	Частично. Нет систематического student-vs-admin negative API/RLS matrix.
Безопасность / производительность	npm audit, Supabase advisors, live RLS metadata, bundle audit.	Нет DAST role matrix, browser XSS payload suite, query-plan SLO или staging load test.

Следующий правильный слой тестов — локальная Supabase/staging integration: две роли и два ученика для проверки RLS/IDOR, конечная проверка сохранения ДЗ/прогресса, а затем ограниченный k6/Locust прогон с фиксированным профилем нагрузки. Не размножать E2E для банков, схем и валидации, которые уже быстро покрыты unit-тестами.

Дефекты: блокируют запуск

1. Ученик может самостоятельно отметить любые уроки пройденными · Critical / P0 — blocker

Шаги	Под ученическим JWT выполнить INSERT/UPDATE в /rest/v1/lesson_progress с собственным user_id, lesson_id предыдущего урока и completed=true; затем вызвать старт итогового теста. В аудите это не эксплуатировалось на аккаунте; возможность подтверждена текущими production RLS-политиками и серверной проверкой unlock.
Expected	Только доверенная серверная операция после валидного завершения должна создавать завершённый прогресс; произвольная запись клиента отклоняется.
Actual	Live policies Users insert own progress / Users update own progress допускают auth.uid()=user_id без ограничения completed; read-only проверка подтвердила для authenticated прямые INSERT=true и UPDATE=true. Серверный assertFinalQuizUnlocked считает такие строки доказательством прохождения 13 уроков. Failing regression test: test_students_cannot_write_course_completion_directly.
Рекомендация	Закрыть прямые INSERT/UPDATE для authenticated (оставить чтение своей записи; сервисную запись выполнять серверным service-role/безопасной функцией); проверить дневной лимит и unlock через двух тестовых пользователей; зафиксировать negative API/RLS tests.

2. Уязвимые runtime-пакеты, включая DOMPurify · High / P1 — до запуска

Шаги	Сверить lockfile и выполнить npm audit --omit=dev; пройти вывод advisory для прямых runtime-зависимостей.
Expected	Продакшн-зависимости без применимых High/Critical advisory; вывод урокового HTML покрыт XSS regression tests.
Actual	Audit нашёл 31 advisory, в том числе High для DOMPurify 3.4.13 и Tiptap/Browserslist/js-yaml зависимостей. DOMPurify реально используется перед dangerouslySetInnerHTML. Известный XSS-advisory DOMPurify ограничен сочетанием IN_PLACE и hook, которого в просмотренной функции не видно; эксплуатируемость именно текущей конфигурации не подтверждена.
Рекомендация	Обновить lockfile до исправленных совместимых версий, проверить release notes; добавить payload regression cases (script/event handlers/javascript URLs/SVG); повторить npm audit и build. Не считать advisory доказанным эксплойтом.

3. В full regression вход ученика показал экран неожиданной ошибки · **Medium / P1 — закрыть до go/no-go**

Шаги	Запустить test_student_can_open_dashboard_and_first_lesson вместе с остальным набором; дождаться marker гидратации /auth.
Expected	Маршрут остаётся гидратированным, ученик попадает в dashboard; ошибок загрузки нет.
Actual	Один раз full suite упал на setup: /auth отрисовал «Что-то пошло не так». Изолированный повтор сразу прошёл (17.4 с). HTTP smoke для /auth возвращал 200. Сбой плавающий, статус причины неизвестен.
Рекомендация	Проверить worker/server logs и browser console за timestamp прогона, записывать correlation/request id; повторить сценарий на staging; считать блокером, если подтверждается на пользовательском трафике или staging.

Дефекты: после запуска / backlog

4. Password leaked protection отключена в Supabase Auth · Medium / P2 — включить перед ростом аудитории

Шаги	Supabase Security Advisor → Auth settings; текущая конфигурация сообщена live advisor.
Expected	Слабые/утекшие пароли отклоняются, особенно при invite/password setup.
Actual	Advisor сообщает, что защита от leaked passwords выключена. Это усиливает риск reuse/credential stuffing, но само по себе не является доказательством захвата аккаунта.
Рекомендация	Включить Password Strength/Leaked Password Protection согласно тарифу; протестировать reset и смену пароля; не журналировать пароль или полный JWT.

5. Не проверена восстановимость бэкапов и файлов Storage · Medium / P2 — до накопления ценных данных

Шаги	Проверить Database → Backups/PITR и выполнить восстановление копии проекта; отдельно проверить резервирование Storage objects.
Expected	Известны RPO/RTO; БД и пользовательские вложения можно восстановить и сверить.
Actual	В этой сессии метаданные бэкапов/restore drill не доступны. Production smoke пропустил приватный bucket check, поскольку не было service-role credentials. Документация Supabase уточняет: DB backups не включают сами Storage-файлы.
Рекомендация	Подтвердить план Supabase, retention и PITR; расписать off-site logical exports для бесплатного тарифа при необходимости; отдельный Storage backup; квартальный restore drill.

6. Frontend-аналитика и сигнализация продуктовых ошибок не обнаружены · Medium / P2 — ближайший продуктовый этап

Шаги	В репозитории отсутствуют SDK/настройки Sentry и PostHog; нет определённой схемы событий обучения.
Expected	Операторы узнают о росте ошибок/недоступности; команда видит шаги курса, drop-off и конверсию до успешного теста.
Actual	Ошибки приходится ловить через пользовательские сообщения и ручной smoke; воронки и retention по событиям продукта не доступны.
Рекомендация	Внедрить Sentry для client/server errors + release health, минимальный ручной event schema в PostHog; не включать autocapture по умолчанию и исключить PII, ответы/текст ДЗ и токены.

7. Нет role-based API/RLS negative integration suite · Medium / P2

Шаги	Запустить проверки попыток читать/менять чужой профиль, ДЗ, группы, прогресс и quiz attempt под student JWT против admin JWT.
Expected	Каждая таблица и server function проверены на owner/admin/unauthenticated границах.
Actual	Тесты в основном статические по SQL и UI smoke; отрицательная RLS матрица на двух аккаунтах отсутствует. Новый progress policy test найденное нарушение сейчас фиксирует как красное.
Рекомендация	Добавить API/integration набор с изолированной Supabase test project; не проводить probing с чужими записями в production.

8. Семь внешних ключей без индексов · Low / P3 — наблюдать и оптимизировать

Шаги	Supabase Performance Advisor перечисляет FK course_invites(created_by, used_by), lesson_block_progress(block_id, lesson_id), quiz_attempts(lesson_id), sql_sandbox_attempts(block_id, lesson_id).
Expected	Частые joins/deletes фильтруют через подходящие индексы при росте данных.
Actual	Advisor сообщает 7 unindexed FK; query plan/slow query data в аудите не получены, поэтому фактическое замедление не измерено.
Рекомендация	Сначала проверить cardinality и реальные запросы/EXPLAIN; добавить только индексы для hot paths и измерить до/после.

Backlog без release-risk

Есть advisor-сообщения о семи неиспользованных индексах и нескольких permissive RLS policies. Не удалять/сливать автоматически: usage может быть низким за период статистики, а policies могут выполнять разные задачи. Отложить до проверки query statistics и RLS matrix.

Технические проверки и ограничения

Проверка	Результат / границы
npm run lint	PASS
npx tsc --noEmit	PASS (до обновления только Python-тестов; TS-файлы не менялись).
npm run build	PASS; client transformed 2,224 modules. Основные generated outputs: app JS ~310 KB; SQL WASM ~658 KB raw (~326 KB gzip); lesson-guide PNG ~1.16 MB raw.
Node unit suite	50 passed / 0 failed.
Python suite	68 passed, 3 skipped, 1 failed (ожидаемый security regression), 1 error (плавающий /auth generic error screen). Общий процесс завершился exit 1.
Isolated repros	Authentication dashboard scenario после error — PASS; mobile homepage сценарий после предыдущего full-run timeout — PASS.
Production smoke	Публичные страницы — 200; retired callback/invite routes — ожидаемые 404; Auth 200; RPC CORS 200; REST без токена 401. Private Storage object/bucket check skipped.
RLS live inventory	19/19 таблиц public имеют RLS включённым. Дополнительно прочитаны policy definitions. RLS включён ≠ политики автоматически безопасны: self-write progress — реальное исключение.
Supabase security advisor	1 authenticated SECURITY DEFINER warning; код функции проверяет private.has_role(auth.uid(), 'admin'), search_path ограничен. Другой alert — leaked password protection disabled.
SQLi / XSS / IDOR	Проверена поверхность кода и политик; пользовательский SQL выполняется локально в seeded SQLite sandbox с allowlist. SQLi не подтверждён. IDOR/self-service progress bypass подтверждён как misconfiguration. Runtime DOMPurify dependency flagged by audit; эксплуатационный payload не отправлялся.
Нагрузка и БД latency	Не выполнялись: для безопасного профиля нужен isolated staging/DB. Нет достоверных p95/p99 API/DB latency или нагрузки, не было измерений production query plans.

Политика SECURITY DEFINER import function имеет grant только authenticated, но внутри сначала проверяет админа и задаёт фиксированный search_path; это предупреждение advisor само по себе не доказывает privilege escalation. По Storage объектам, приватным бакетам и restore backup необходим отдельный staging/секретный service-role smoke.

Минимальный разумный стек улучшений

Когда	Инструмент	Что включить
Сейчас	Sentry (TanStack Start React)	Клиентские + server-function exceptions, release/environment, source maps, performance samples. Redact auth headers, query/body и student PII; сначала включить только ошибки и health.
Сейчас	PostHog Product Analytics	Только ручные события: course_started, lesson_started/completed, block_completed, homework_submitted/reviewed, final_quiz_started/passed/failed, course_completed. Воронка enrol/start → уроки 1–13 → итоговый тест → course complete.
Сейчас	Uptime / synthetic checks + существующие Supabase/Cloudflare dashboards	Проверять landing/auth/admin-login публично, отдельным тестовым аккаунтом — вход и чтение урока. Алерты на availability/error rate, но не выводить персональные ответы/контент.
Сейчас	Структурированные logs	Добавить request/correlation id, route/function, duration_ms, status/error class; редактировать email, JWT, пароль, файлы и ДЗ. Хранить ограниченный retention.
Позже / рост	OpenTelemetry + Grafana Cloud/Prometheus	Когда понадобится корреляция frontend/API/DB, SLO и p95/p99; не строить self-hosted Grafana stack на текущем масштабе без эксплуатационной причины.
Позже	AI-помощник курса	Только после аналитики и privacy review: ответы по утверждённым материалам, явные границы, контроль стоимости, feedback/evaluation set; не передавать стороннему LLM профили, ответы теста и тексты ДЗ.

Метрики обучения, которые стоит определить до установки SDK

- Воронка: первый вход → старт урока → завершение 1–13 → старт теста → pass → завершение курса.
- Drop-off по уроку и блоку: started/completed events с lesson_id, block_id/type и course cohort; сравнивать когорты, не людей.
- Время урока: elapsed time между стартом/завершением с учётом visibility/idle; считать диагностической метрикой, а не рейтингом ученика.
- Сложные вопросы: агрегировать частоту неверных ответов по question_id/topic, не экспортируя конкретный ответ.
- Retention: возвращение по неделе когорты; сначала определить, что означает «активный» пользователь для 14-дневного курса.

У PostHog есть встроенные funnels и retention insights; OpenTelemetry задаёт переносимую модель traces/metrics/logs. Supabase DB backups не включают сами Storage objects — план восстановления должен охватывать оба контура.

Источники и следующие действия

Ссылки ниже использованы для проверки рекомендаций и advisory remediation. Архитектурные выводы и результаты проекта основаны на репозитории и live read-only проверках.

- Supabase Security Advisor: [SECURITY DEFINER advisory](#)
- Supabase password security: [leaked-password protection](#)
- Supabase backups: [database backups and PITR](#)
- PostHog: [funnels](#) и [retention](#)
- TanStack Start observability: [Sentry/structured logging patterns](#)
- OpenTelemetry: [signals — traces, metrics, logs](#)
- Sentry TanStack Start: [performance/span metrics](#)

Что зафиксировано в рабочей копии

Добавлен автотест-контракт `autotests/unit/test_progress_access_policy.py`, который краснеет на текущих разрешениях. Для flaky mobile homepage smoke добавлен одиночный повтор навигации. Новая failing проверка намеренно не помечена `skip/xfail`: она должна оставаться видимой до исправления политик. Код не коммитился и не публиковался.

Критичный следующий шаг: ограничить клиентские записи `lesson_progress`, затем прогнать полный regression повторно в `disposable/staging` среде и проверить RLS матрицу. Только после этого имеет смысл финализировать `go/no-go`.